SHOPEZ

E-Commerce Web Application

Project Documentation

Full-Stack Web Application
React • Node.js • Express.js • MongoDB • Mongoose

1. INTRODUCTION

SHOPEZ is a full-stack e-commerce web application designed to provide a complete online shopping experience. Users can browse products, view product details, add products to a shopping cart, complete checkout, place orders, and view their order history.

The application uses a React frontend, a Node.js and Express.js backend, MongoDB with Mongoose for persistence, Axios for API communication, React Router for navigation, and JWT-based authentication for protected functionality.

2. SCENARIO BASED CASE STUDY

Background

SHOPEZ is designed for customers who want a simple and accessible platform for discovering products and completing purchases online. Administrators are provided with tools for managing products and customer orders.

Scenario

A customer visits SHOPEZ to browse the available catalog. The customer can open a product, review its details, select a quantity, and add it to the cart. After authentication, the customer can proceed to checkout, provide shipping information, place the order, and view the order later through My Orders.

Challenge

- Provide an easy-to-use online shopping interface.
- Support secure user authentication.
- Allow customers to manage a shopping cart.
- Provide checkout and order creation.
- Allow customers to review their orders.
- Provide administrators with product and order management.

Solution

SHOPEZ addresses these requirements through a React frontend, REST APIs built with Express.js, MongoDB persistence, JWT authentication, protected routes, and separate customer and administrator workflows.

3. SYSTEM REQUIREMENTS

3.1 Software Requirements

- Windows 10/11, macOS, or Linux.
- Node.js and npm.
- React.js with Vite.
- Express.js.
- MongoDB and Mongoose.
- Axios.
- React Router.
- JWT authentication.
- Visual Studio Code.
- Google Chrome or another modern browser.

3.2 Hardware Requirements

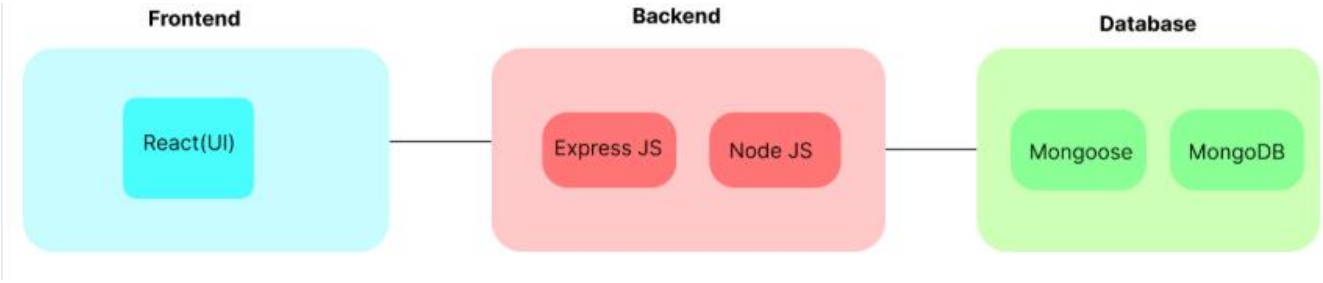
- Intel Core i5 / AMD Ryzen 5 or equivalent or better.

- Minimum 8 GB RAM; 16 GB recommended.
- Sufficient free storage for dependencies, MongoDB data, and project files.
- 1366×768 or higher display recommended.

4. TECHNICAL ARCHITECTURE

SHOPEZ follows a three-layer web application architecture. The React frontend provides the user interface, the Node.js/Express.js backend exposes REST APIs and application logic, and MongoDB stores persistent application data through Mongoose.

```
React Frontend
      │ HTTP / REST API
      ▼
Node.js + Express.js Backend
      │ Mongoose
      ▼
MongoDB Database
```



SHOPEZ technical architecture supplied with the project resources.

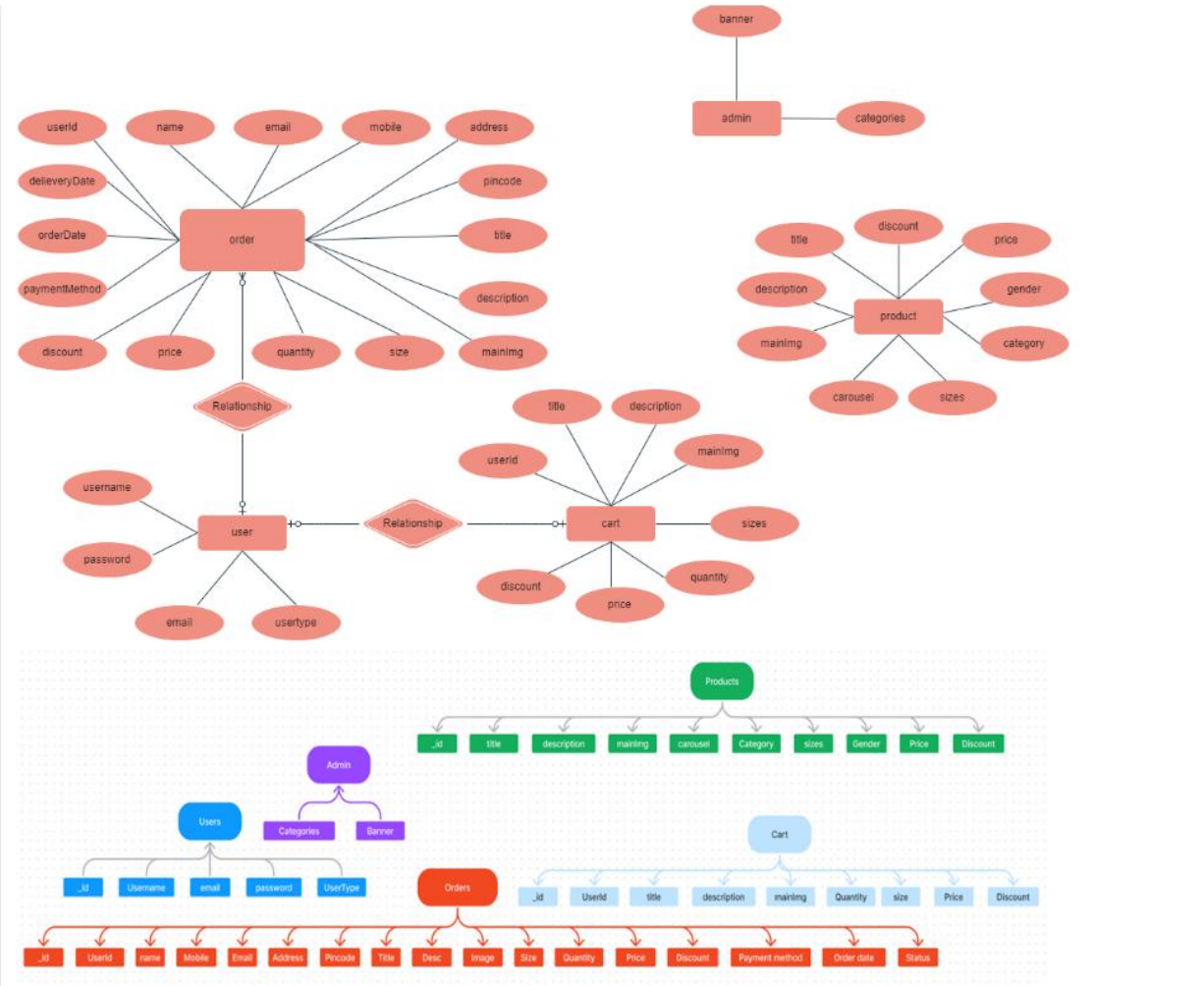
5. ER DIAGRAM

The project resources provide an ER/data model covering users, products, carts, orders, and administrator-related information.

Implemented Data Models

- User: name, email, password, and role.
- Product: name, description, price, image, category, stock, and rating.
- Order: user reference, ordered items, shipping address, total price, and order status.

The supplied ER diagram contains additional fields/entities from the project resource. The implementation description above reflects the current application models.



SHOPEZ ER/data model supplied with the project resources.

6. KEY FEATURES

User Features

- Registration and login
- JWT authentication
- Protected routes
- Product browsing
- Product details
- Stock-aware cart quantity management
- Shopping cart
- Checkout and shipping address
- Order placement
- My Orders
- Logout

Administrator Features

- Protected admin dashboard
- Add products
- Edit products

- Delete products
- View orders
- Update order status

7. ROLES AND RESPONSIBILITIES

User

- Register and log in
- Browse products
- View product details
- Manage cart items
- Provide checkout/shipping information
- Place orders
- View previous orders
- Log out

Admin

- Access the protected dashboard
- Create products
- Update products
- Delete products
- Review customer orders
- Update order status

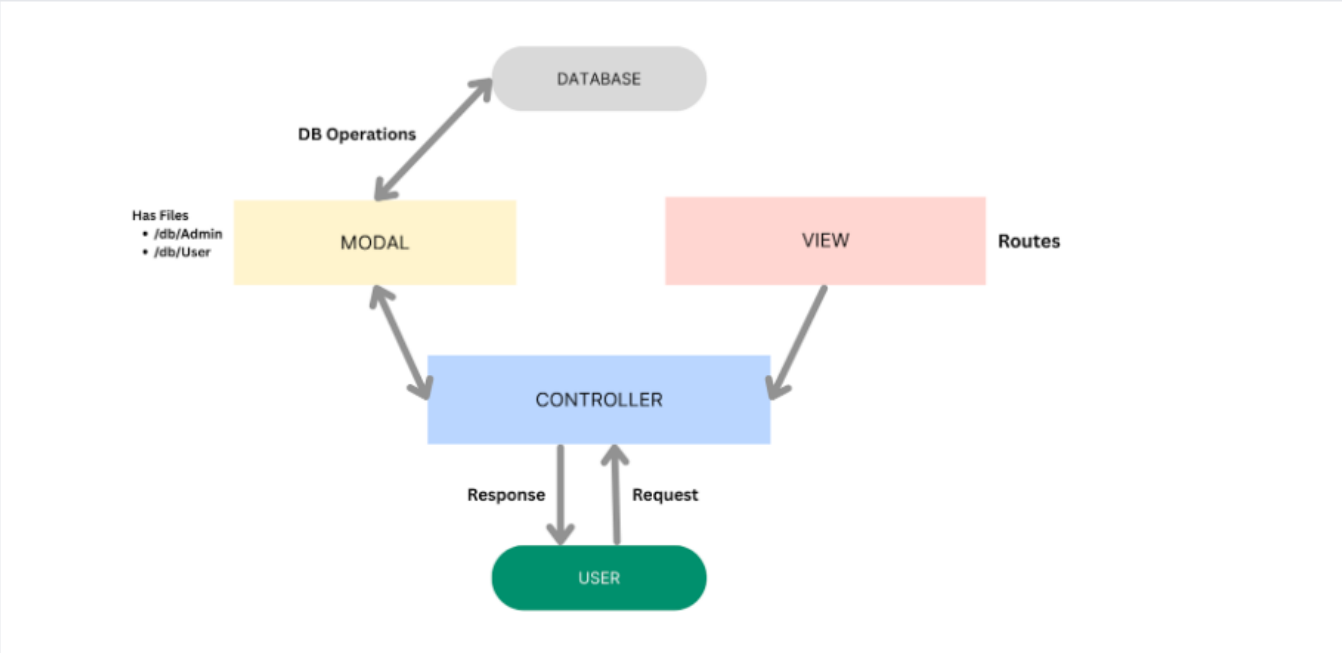
8. USER FLOW

```
START
  ↓
Home / Product Browsing
  ↓
Register / Login
  ↓
Products
  ↓
Product Details
  ↓
Add to Cart
  ↓
Cart
  ↓
Checkout
  ↓
Shipping Information
  ↓
Place Order
  ↓
My Orders
  ↓
END
```

The administrator workflow begins with authorized admin access to the dashboard, followed by product management and order-status management.

9. MVC PATTERN

The SHOPEZ backend follows the Model-View-Controller (MVC) architectural pattern. This separates database models, request-processing logic, and routing responsibilities.



MVC pattern supplied with the SHOPEZ project resources.

Model Layer

The Model layer defines application data schemas and database operations using Mongoose and MongoDB.

Controller Layer

The Controller layer receives requests, validates and processes input, communicates with models, and returns responses.

Routing Layer

The routing layer defines REST endpoints and connects requests to controller functions.

```
models/       User.js | Product.js | Order.js
controllers/  authController.js | productController.js | orderController.js
routes/       authRoutes.js | productRoutes.js | orderRoutes.js
```

Advantages

- Separation of concerns
- Maintainability
- Scalability
- Reusability
- Independent testing

10. PROJECT SETUP AND CONFIGURATION

Project Structure

```
SHOPEZ/
├── client/
└── server/
```

Frontend Setup

```
cd client
npm install
npm run dev
```

Backend Setup

```
cd server
npm install
npm run dev
```

The backend requires the configured MongoDB connection and JWT secret environment variables. Environment files are excluded from the public repository.

11. BACKEND DEVELOPMENT

Backend Structure

```
server/
├── controllers/
├── models/
├── routes/
├── middleware/
└── server.js
```

The backend provides authentication, product, and order APIs. It validates requests, communicates with MongoDB, applies authorization rules, and calculates order totals on the server.

Authentication

Registration creates a user account with a securely hashed password. Login validates credentials and returns a JWT. Protected endpoints use the JWT to identify the authenticated user.

Product API

Public endpoints retrieve products and individual product details. Administrator-protected endpoints create, update, and delete products.

Order API

Authenticated users can create orders and retrieve their own order history. Administrators can retrieve orders and update order status. Order totals are calculated by the server using current product information.

12. DATABASE DEVELOPMENT

MongoDB is used as the persistent database and Mongoose is used as the Object Data Modeling layer.

- Users: customer/admin identity and authentication information.
- Products: product catalog information, price, stock, category, image, and rating.
- Orders: customer orders, items, quantities, prices, shipping address, total price, and status.

Orders reference the user who placed them and contain product references within their items. The order controller validates stock, calculates the total price, reduces stock, and creates the order.

13. FRONTEND DEVELOPMENT

```
client/src/
├── components/
│   ├── Navbar.jsx
│   ├── ProductCard.jsx
│   └── ProtectedRoute.jsx
├── context/
│   ├── AuthContext.jsx
│   └── CartContext.jsx
├── pages/
│   ├── Home.jsx
│   ├── Login.jsx
│   ├── Register.jsx
│   ├── ProductDetails.jsx
│   ├── Cart.jsx
│   ├── Checkout.jsx
│   ├── Orders.jsx
│   └── Admin.jsx
├── api.js
├── App.jsx
└── main.jsx
```

- AuthContext: global authentication state.
- CartContext: global cart state and quantity management.
- ProtectedRoute: access control for authenticated/admin pages.
- ProductCard: reusable product display.
- ProductDetails: individual product information and quantity selection.
- Checkout: shipping information and order creation.
- Orders: authenticated user's order history.
- Admin: product and order management.

14. PROJECT EXECUTION

```
Terminal 1
cd server
npm run dev

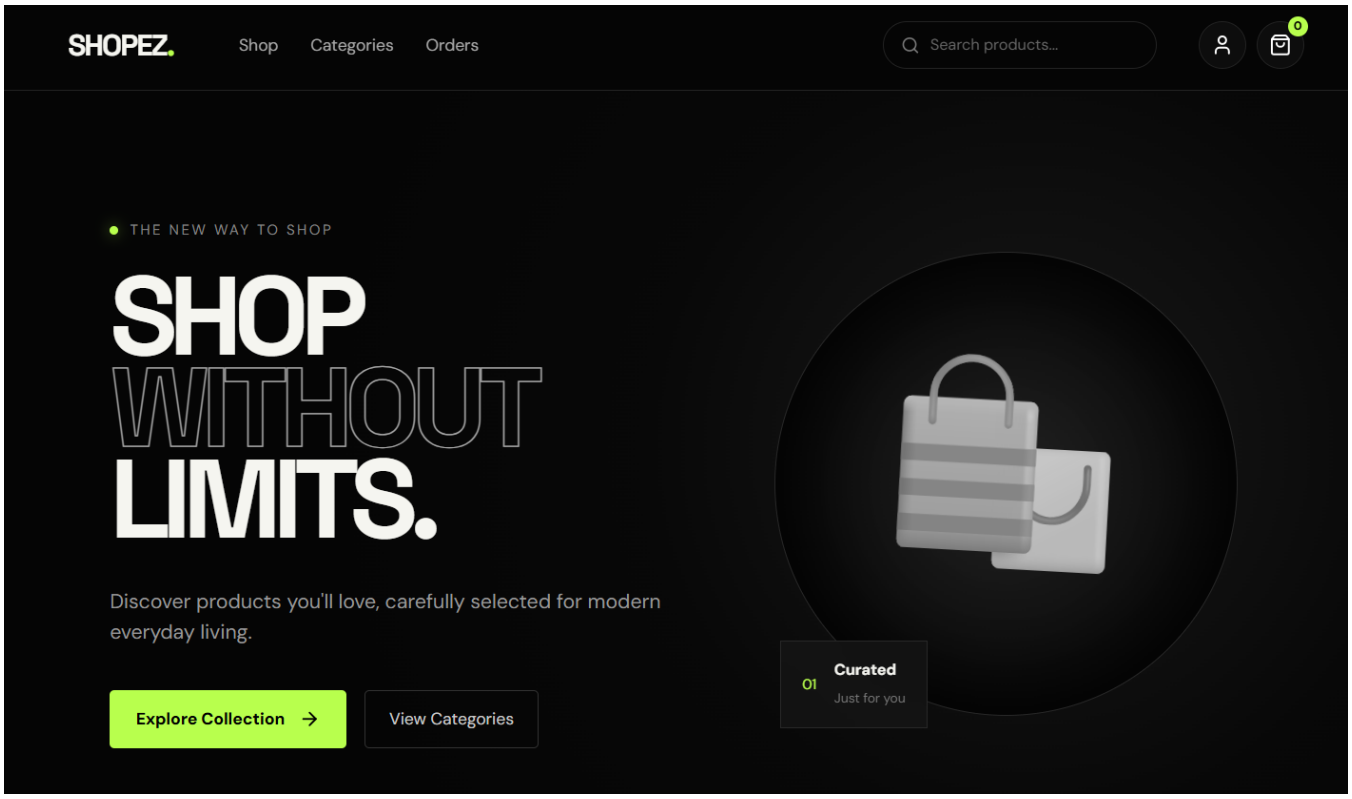
Terminal 2
cd client
npm run dev
```

- Start MongoDB.
- Start the Express backend.
- Start the Vite frontend.
- Open the application in a browser.
- Demonstrate authentication, products, cart, checkout, orders, and admin functionality.

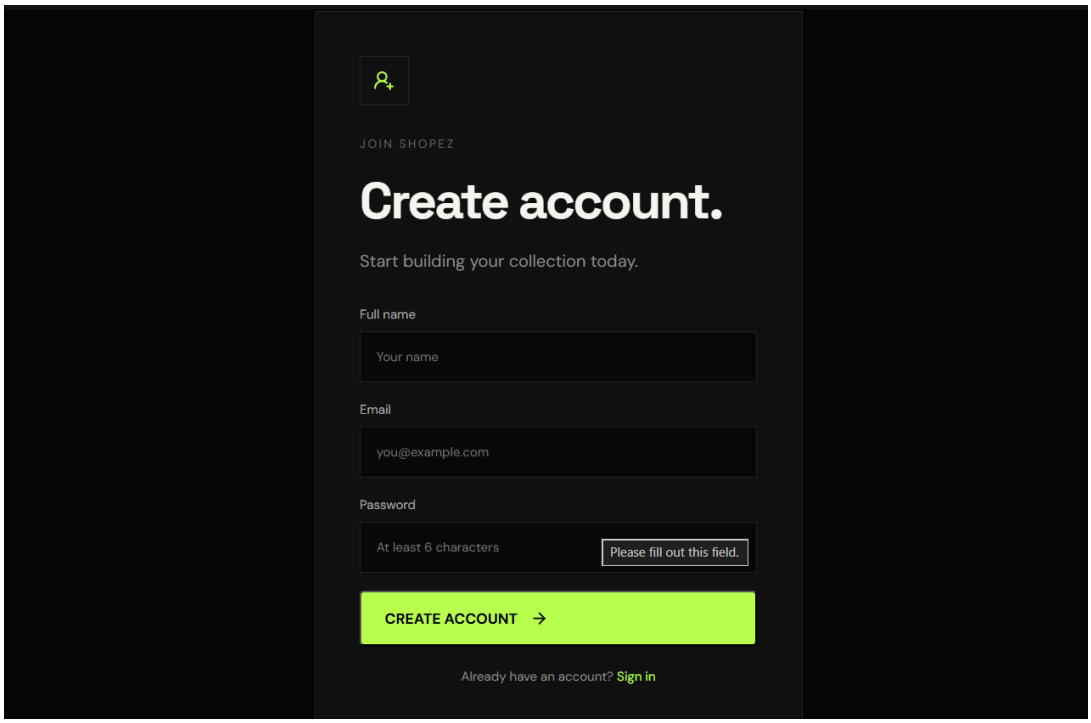
15. OUTPUT SCREENSHOTS

Insert screenshots from the running SHOPEZ application for the following screens:

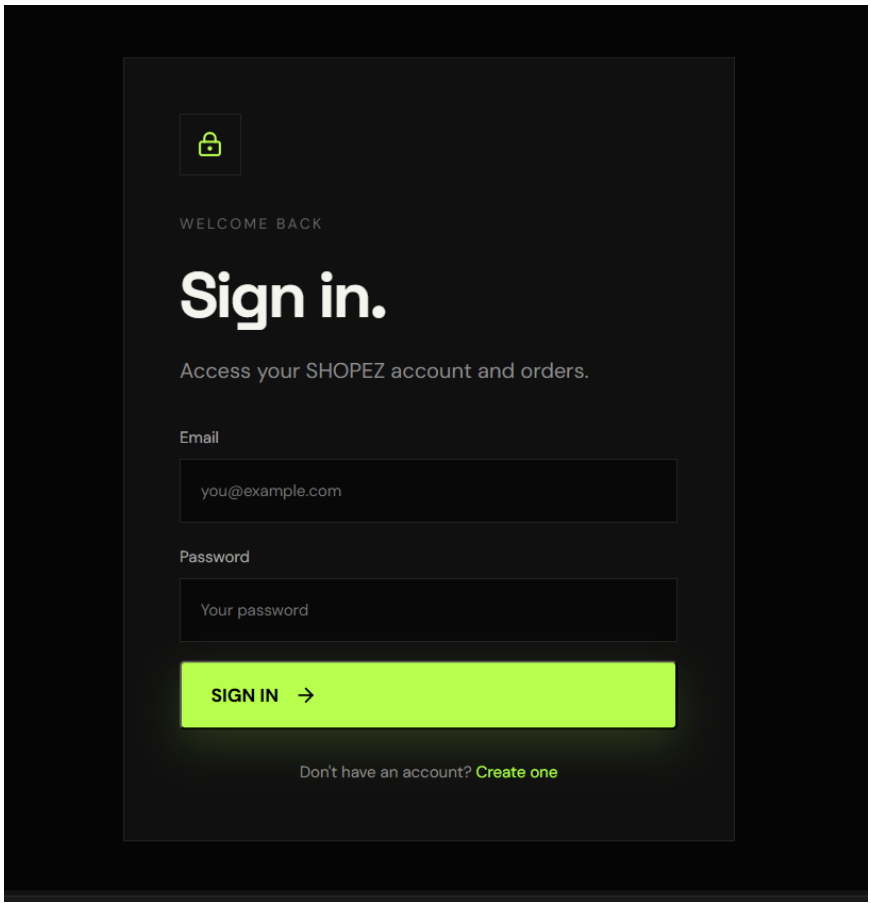
- Home:



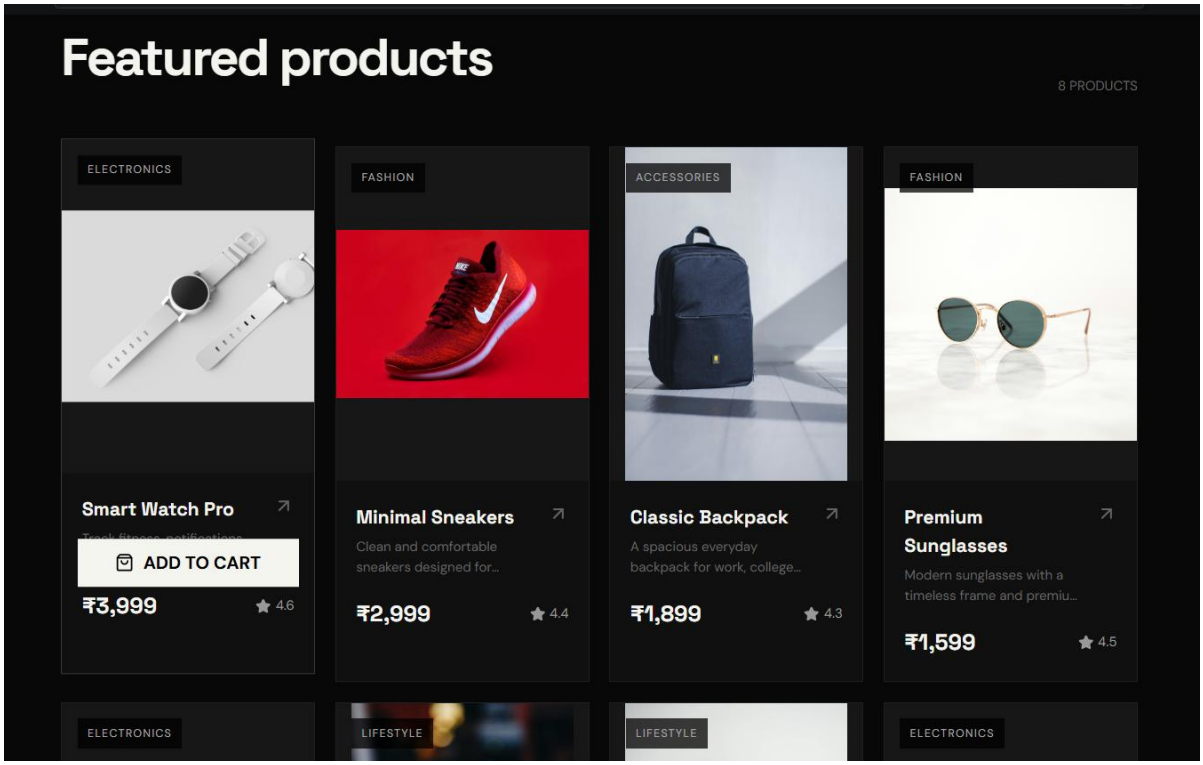
- Register:

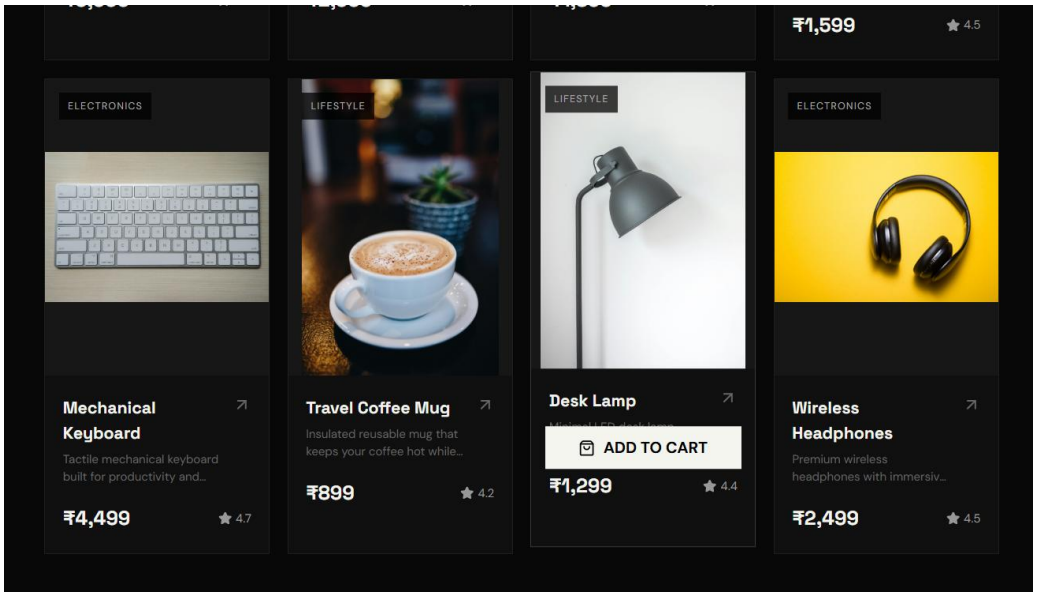


- Login:

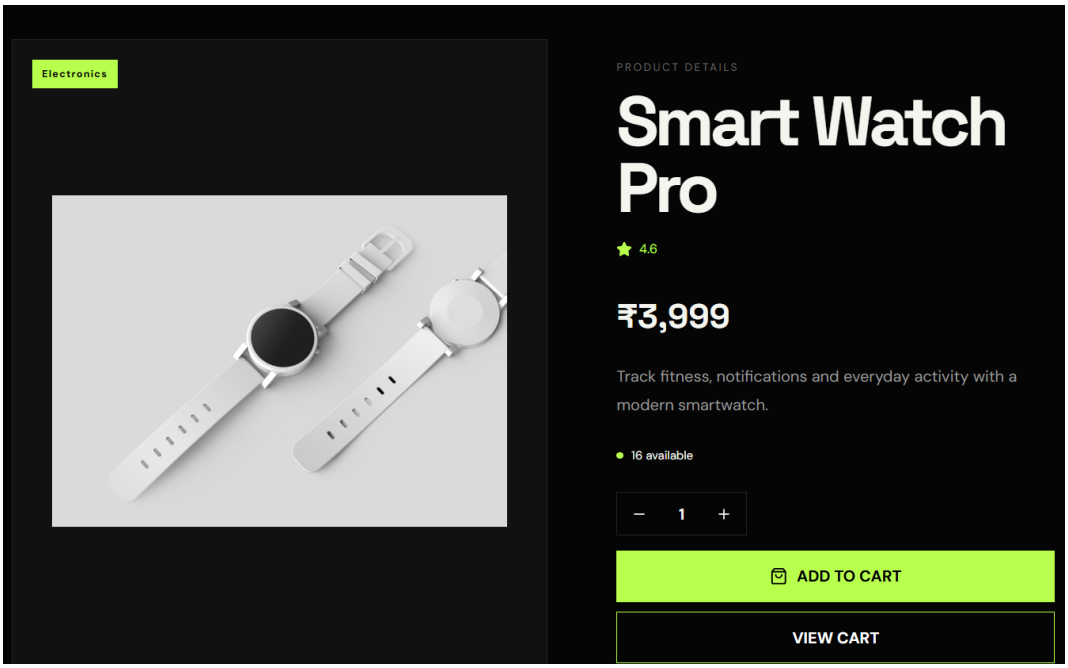


- Product Listing:

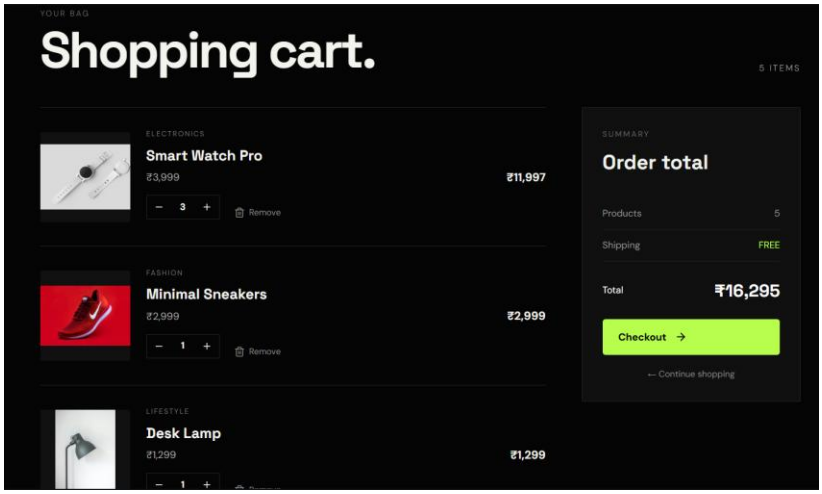




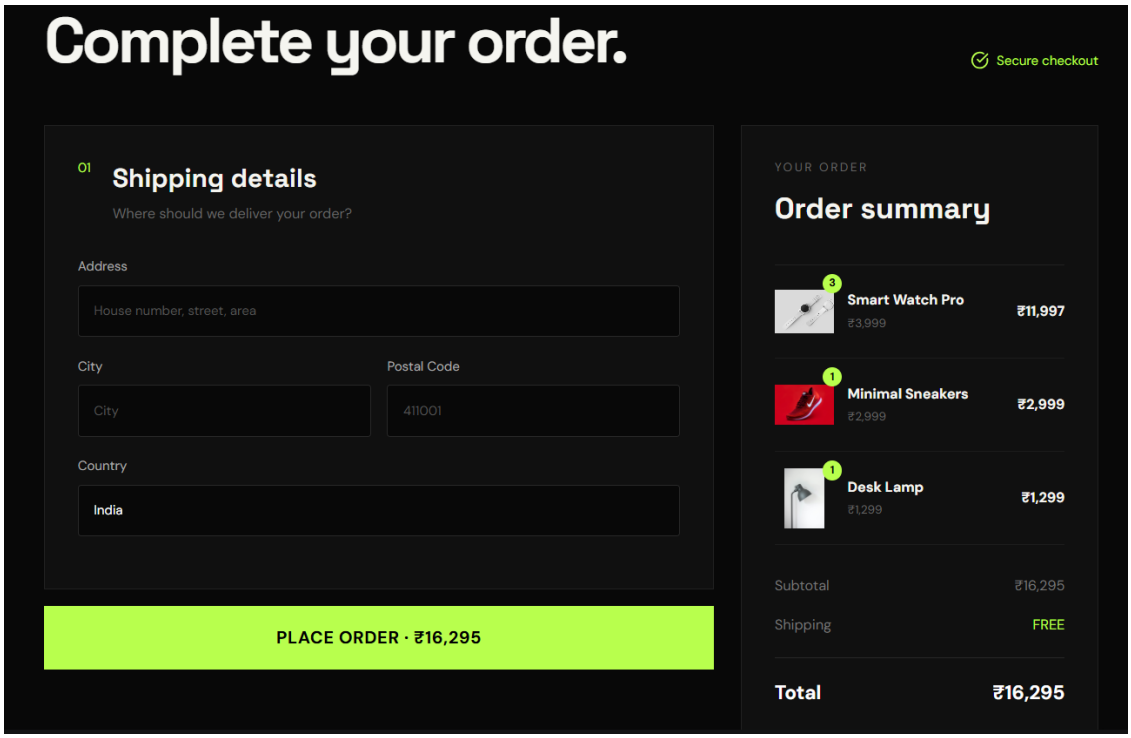
- Product Details:



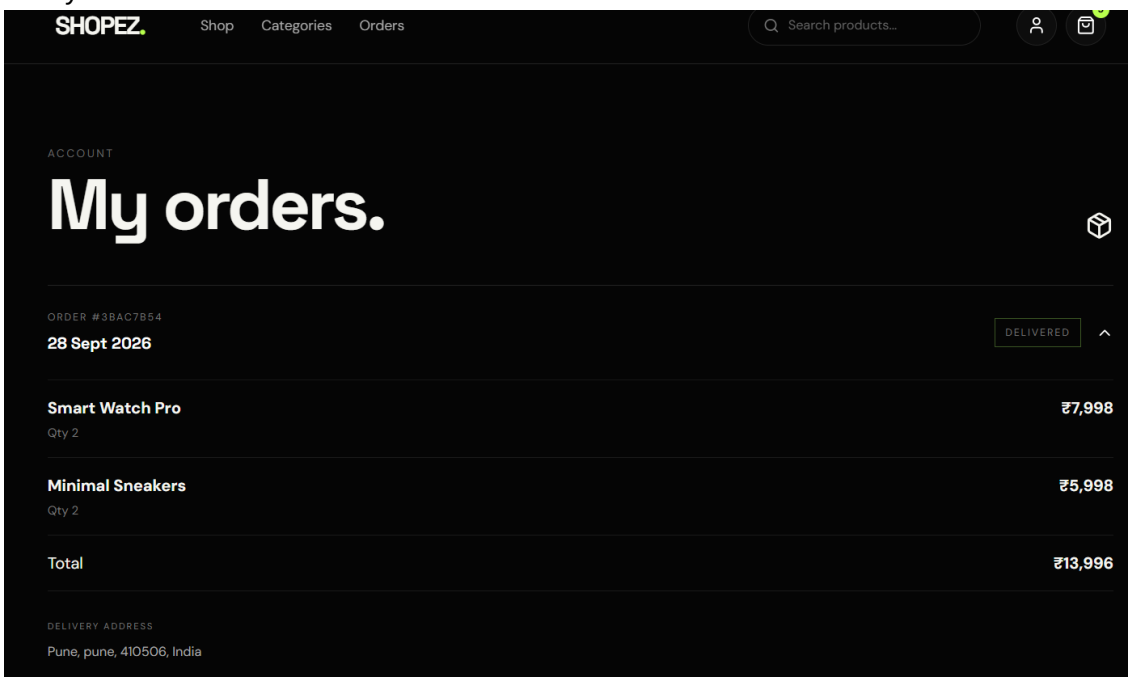
- Cart:



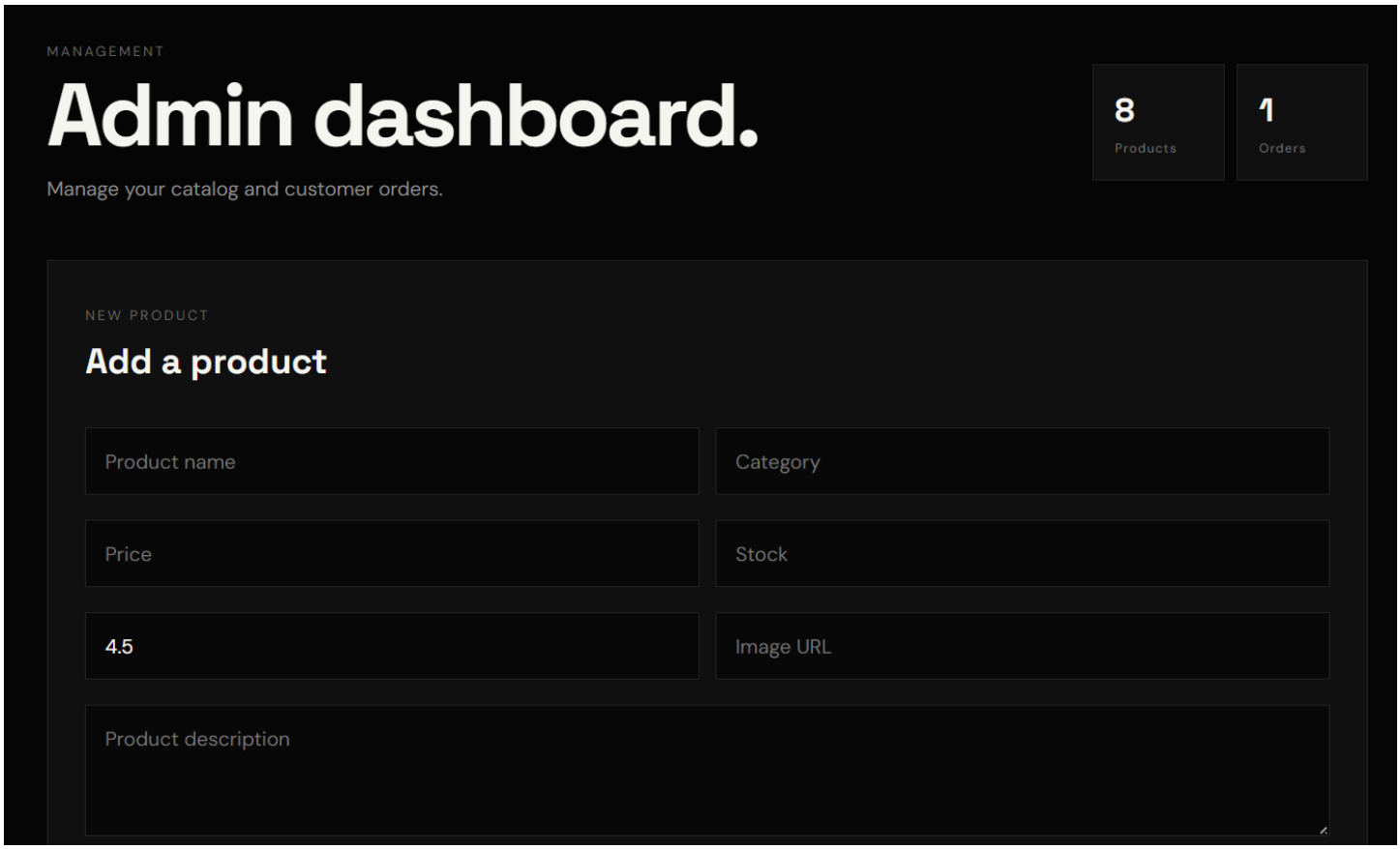
- Checkout:



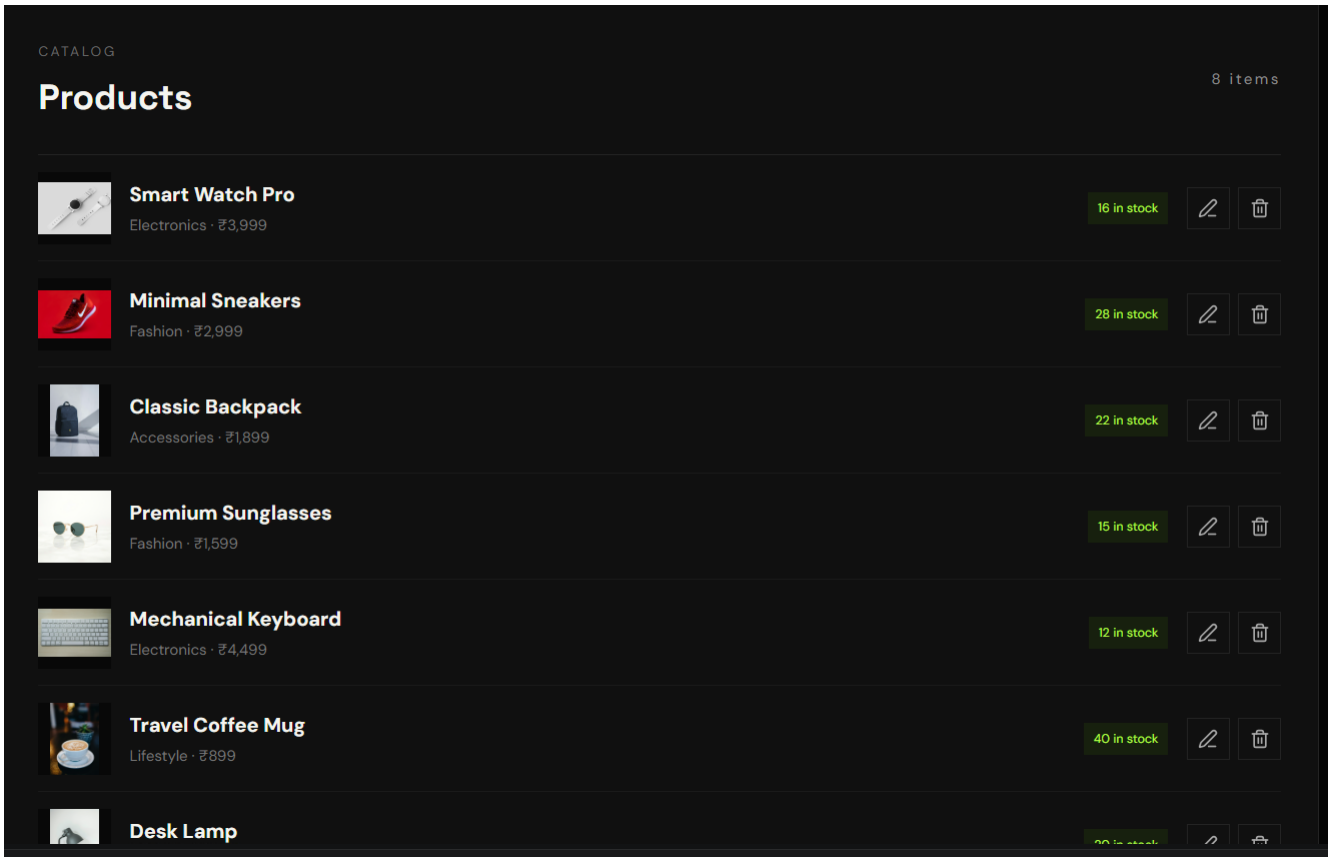
- My Orders:



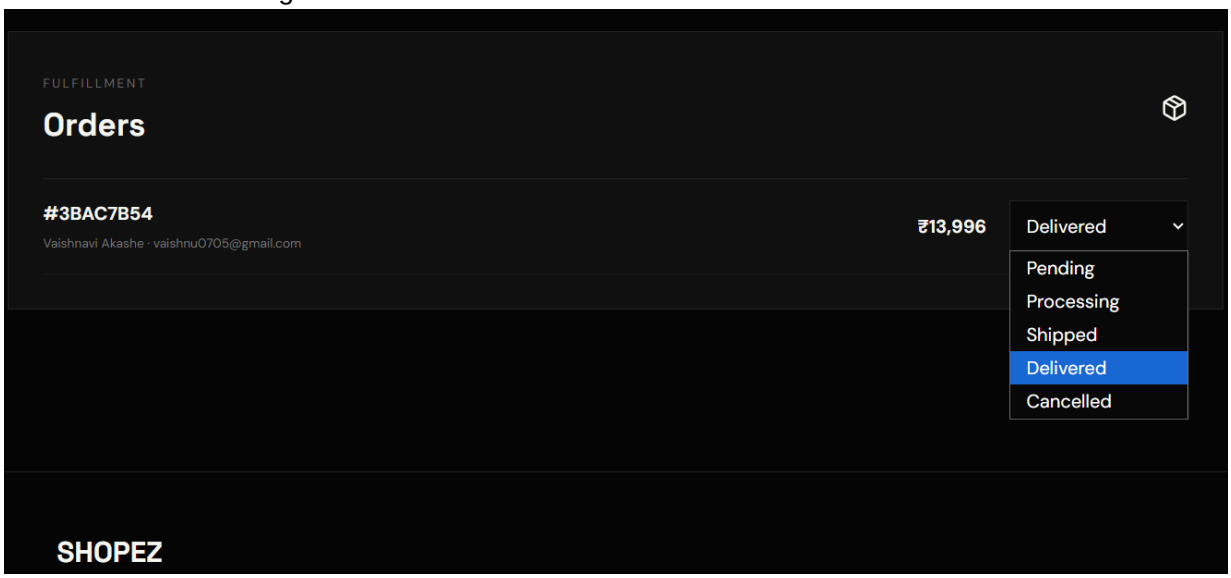
- Admin Dashboard:



- Admin Product Management:



- Admin Order Management:



16. DEMO VIDEO LINK

Demo Video Title: SHOPEZ – Project Explanation and Demonstration

Demo Video Link: https://drive.google.com/file/d/1Mh3RCpv_rTLh6u6dUW-Qeq_EiHY_3KO_/view?usp=drive_link

17. CODE REPOSITORY LINK

GitHub Repository: https://github.com/Vaishnaviakashe/SHOPEZ_E_commerce

The repository contains the SHOPEZ client and server source code and project configuration.